

A1 Retrospective: Object Models Under Concurrency

A1 Retrospective

[photo 1516321318423 f06f85e504b3?w=1200] | <https://images.unsplash.com/photo-1516321318423-f06f85e504b3?w=1200>

What We Learned

- Object ownership matters
- Concurrency exposes design flaws
- Invariants drive architecture
- AI recommendations require validation
- Thread-safe is not the same as correct

NOTE

The goal of A1 was not to teach locking. The goal was to teach object-model correctness under concurrency.

Assignment Goals

What A1 Was Designed To Teach

Topic	Purpose
Object Model	Define ownership, responsibilities, and boundaries
Concurrency	Expose weaknesses in object interactions
Invariants	Define what must always remain true
Failure Analysis	Understand why correct-looking systems fail
Redesign	Preserve correctness under concurrent execution
AI Critique	Validate rather than blindly trust recommendations

Class Performance Summary

Aggregate Assessment

Skill Area	Score
Concurrency Basics	95
Race Identification	94
Two-Stack Implementation	92
OO Modeling	90
Invariant Thinking	82
Ownership Modeling	80
Tradeoff Analysis	76
Architectural Reasoning	75
AI Critique	68

NOTE

The class is strong technically.

The biggest growth opportunities are: ownership, architecture, and AI critique.

What Students Did Well

Consistent Strengths

- All teams demonstrated real concurrency
- All teams produced observable failures
- Most teams generated useful logs
- Most redesigns restored correctness
- Two-stack implementations were generally strong
- Students understood race conditions

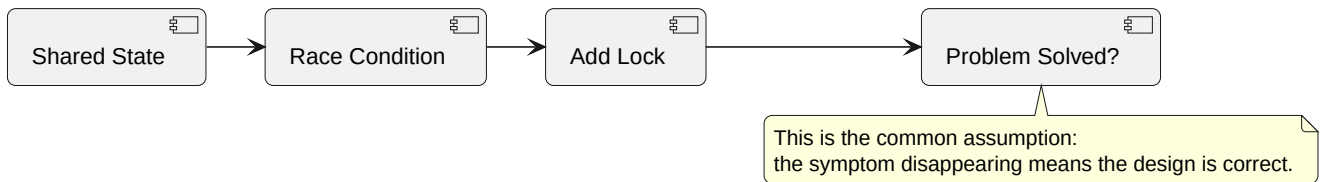
Overall

The class understands concurrency.

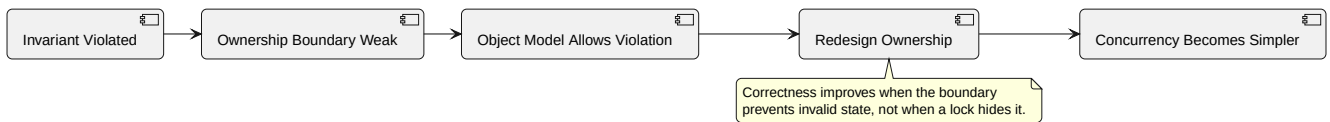
The next step is understanding architecture.

Common Failure Pattern

Student Thinking



Architectural Thinking



Deep Dive #1

Ownership vs Synchronization

[photo 1451187580459 43490279c0fa?w=1200] | <https://images.unsplash.com/photo-1451187580459-43490279c0fa?w=1200>

Key Question

Instead of:

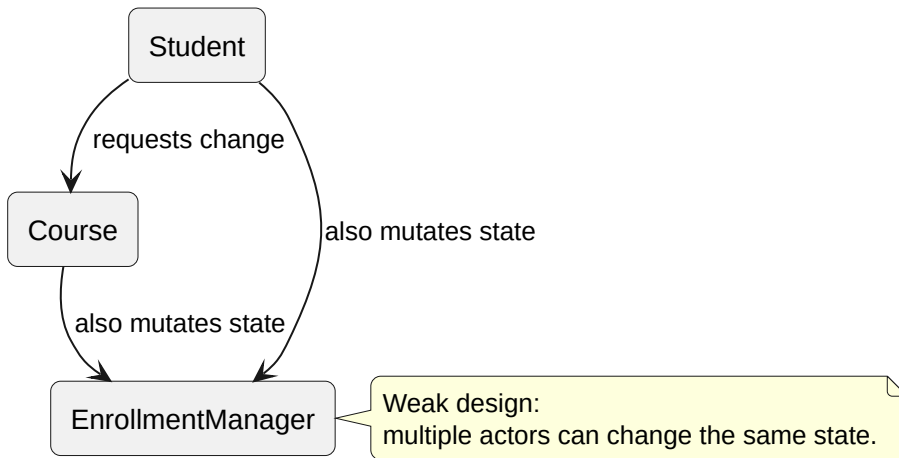
```
synchronized(course)
```

Ask:

Why can multiple threads modify Course?

Ownership Analysis

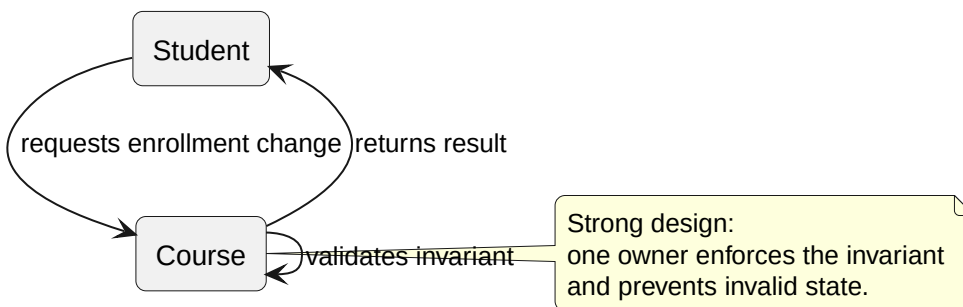
Weak Design



Everyone modifies enrollment.

Nobody owns enrollment.

Strong Design



Course owns enrollment.

Course enforces invariants.

Ownership Findings

What We Observed

Pattern	Observation
Shared State	Students identified it successfully
Ownership	Often unclear
Authority	Rarely documented
Boundary Enforcement	Usually implemented with locks
Ownership Reduction	Rarely considered

Deep Dive #2

Invariants Drive Architecture

[photo 1520607162513 77705c0f0d4a?w=1200] | <https://images.unsplash.com/photo-1520607162513-77705c0f0d4a?w=1200>

Typical Pattern

Invariant:
Enrollment $\leq$ Capacity

Fix:
Add Lock

Better Pattern

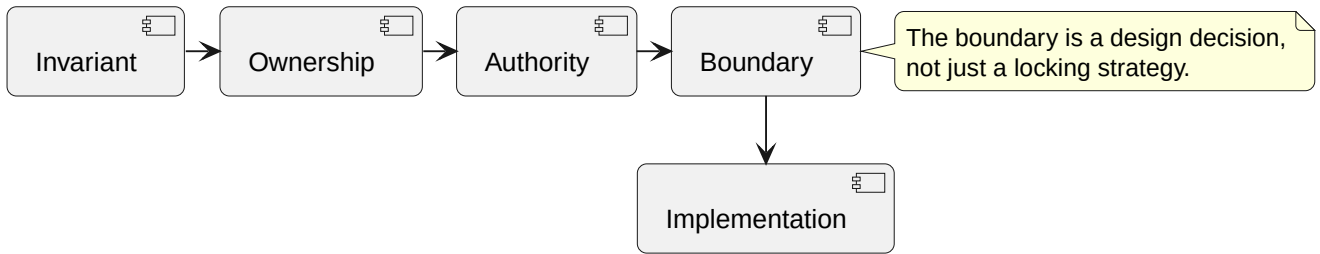
Invariant:
Enrollment $\leq$ Capacity

Therefore:
Course owns enrollment decisions

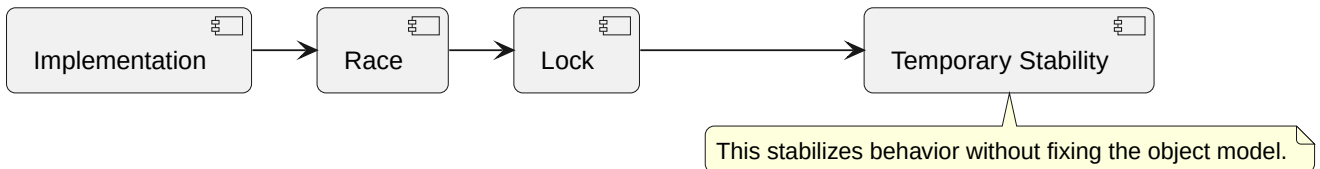
Invariant Driven Design

Strong Design Flow

Invariant
Ownership
Authority
Boundary
Implementation



Weak Design Flow



Invariant Findings

Observations

What Students Did	What Was Missing
Listed invariants	Used invariants to justify architecture
Identified violations	Derived ownership from invariants
Validated correctness	Explained authority boundaries
Added synchronization	Reduced shared state

Deep Dive #3

AI Is Not An Architect

[photo 1677442136019 21780ecad995?w=1200] | <https://images.unsplash.com/photo-1677442136019-21780ecad995?w=1200>

Largest Weakness Across Submissions

Most critiques focused on:

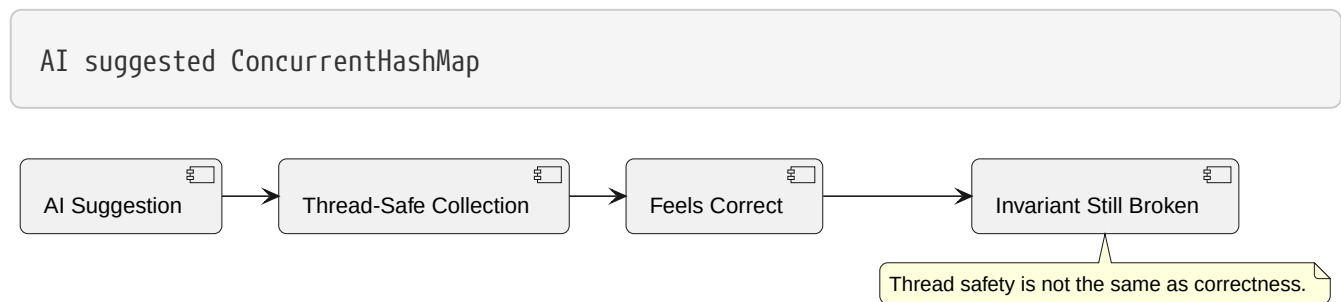
- Locks
- Synchronization
- Thread-safe collections

Few critiques focused on:

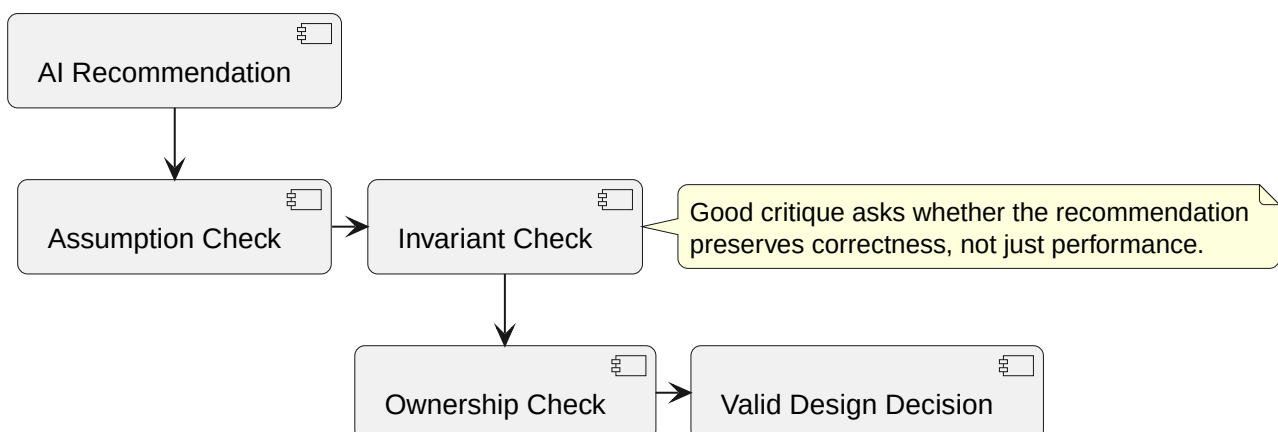
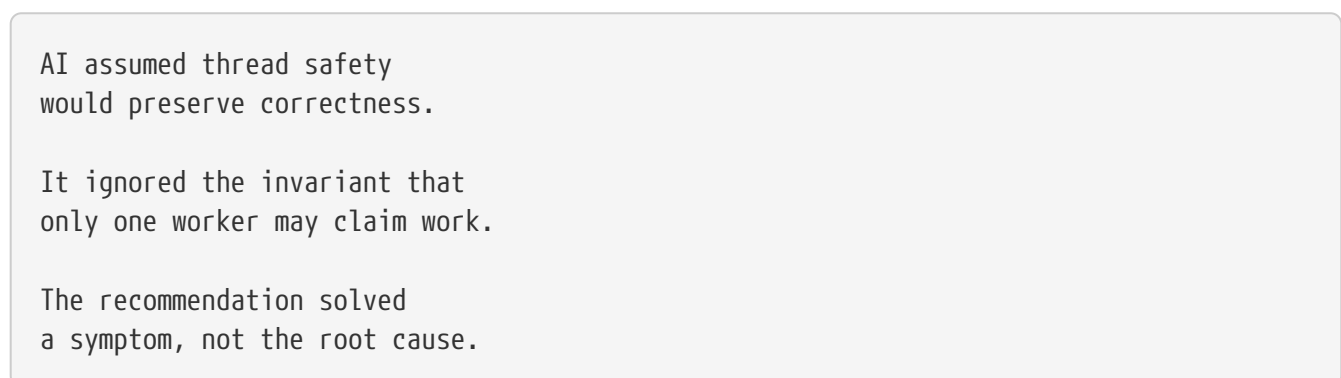
- Assumptions
- Ownership
- Invariants
- Architectural consequences

Weak vs Strong AI Critique

Weak Critique



Strong Critique



AI Review Checklist

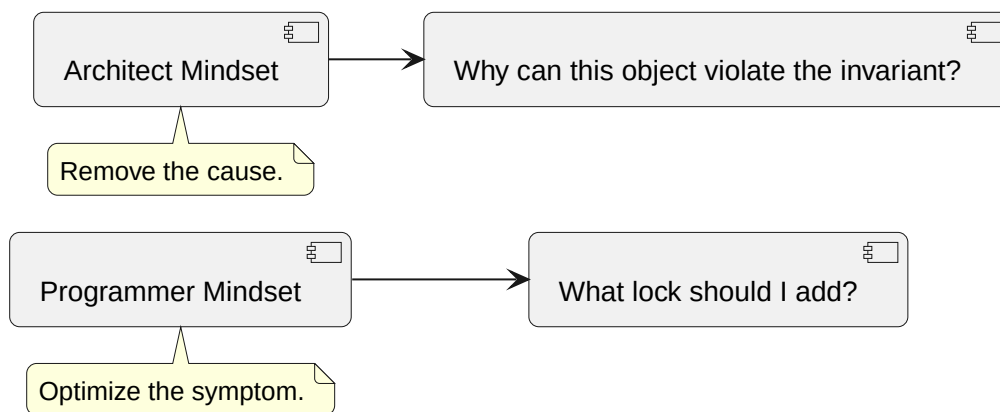
Before Accepting AI Recommendations

- What assumptions is AI making?
- What invariant is AI protecting?
- What invariant is AI ignoring?
- Who owns the state?
- Can ownership remove sharing?
- What tradeoffs exist?
- How will we validate correctness?

Architecture vs Programming

Programmer Mindset

What lock should I add?



Architect Mindset

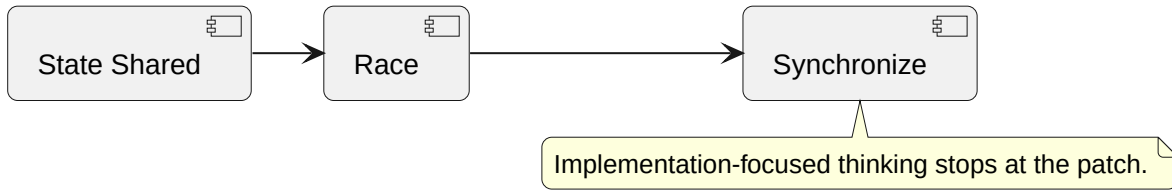
Why does this object permit
the invariant to be violated?

Cross-Submission Findings

Group 1: Implementation Focused

- Aircraft Tracking
- Course Enrollment

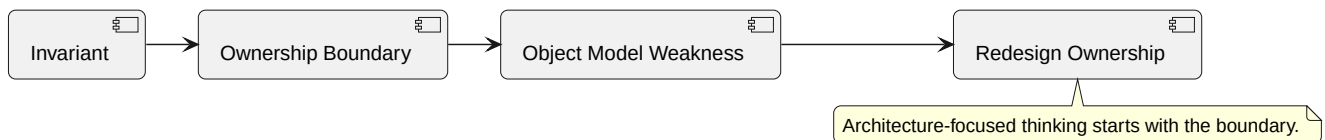
Pattern:



Group 2: Architecture Focused

- Commodities Exchange
- Context OS
- Content Package

Pattern:



Class-Wide Opportunities

Areas To Improve

Strengths	Growth Areas
Concurrency Basics	Ownership Modeling
Race Detection	Architectural Reasoning
Logging	Tradeoff Analysis
Validation	AI Critique
Two-Stack Development	Invariant-Driven Design

What We Need For A2

Focus Areas

- Ownership before synchronization
- Invariants before implementation
- Authority before access
- Architecture before optimization
- Validation before trust

- AI as advisor, not architect

Final Takeaway

[photo 1516321497487 e288fb19713f?w=1200] | <https://images.unsplash.com/photo-1516321497487-e288fb19713f?w=1200>

Most Important Lesson

Concurrency did not create the problem.

Concurrency exposed a problem that already existed in the object model.

Remember

- Clear ownership simplifies concurrency
- Invariants drive architecture
- Thread-safe does not mean correct
- AI recommendations require validation
- Architecture is about preserving correctness